Image Processing - Exercise 4

Shay Morad

Introduction

In this exercise, the primary goal is to implement the "Stereo Mosaicing" algorithm. This algorithm generates a sequence of panoramas from a video that is scanning a scene via horizontal/vertical camera movement (left-to-right or right-to-left for example). The main concept involves computing rigid transformations between consecutive frames, assuming significant overlap. The primary techniques used include keypoint detection, feature matching, estimation of affine transformations using RANSAC and lastly mosaics creation using different strips from frames. This implementation aims to support both dynamic and changing viewpoint mosaics.

Algorithm

1. Overview

   The proposed algorithm is structured as follows:

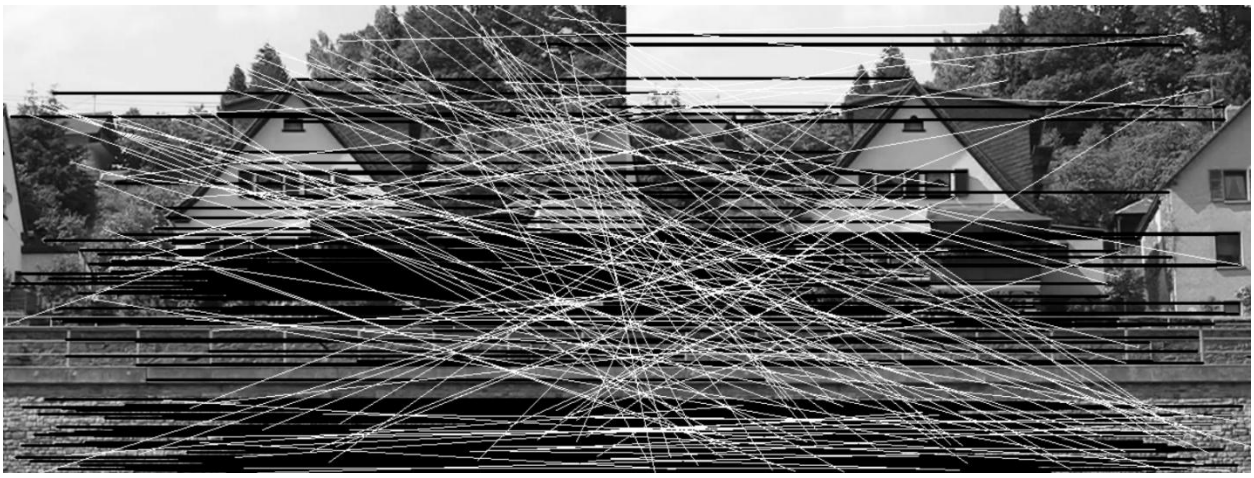
   a. Frame Preprocessing: The video frames are loaded and converted to grayscale. (Optional: reduce video resolution, fps, and more..)
   b. Transformation Estimation: Forward transformations between consecutive frames.
   c. Cumulative Transformation Calculation (Composite Transformation): $Frame_i$ to $Frame_m$ transformations are computed using matrix composition and inversions if needed.
   d. Mosaics Video Creation: Panoramas are created by adding vertical strips from consecutive frames. At first, the panoramas are empty canvases. The initial canvas size is calculated as suggested in class, by applying the respective composite transformation of $Frame_i$ (for each frame), on its corners, and then choosing the minimum and maximum distance corners to calculate the canvas. Strips are dynamically taken from each frame, by respect to that frame's X translation compared to the reference frame. Just as suggested in class.

2. Sub-Algorithm Descriptions

2.1 Keypoint Detection and Feature Matching

- Input: Two consecutive frames.
- Output: Matched keypoint pairs.

Visualization:



The matched keypoints and inliers are visualized with lines connecting corresponding points (e.g., white lines for outliers, black lines for inliers).

2.2 Transformation Estimation

- Input: Keypoints from matched feature pairs.
- Output: 2D affine transformation matrix.

Visualization:

```
Source frame keypoints: [[276. 168.]]
Target frame keypoints: [[273. 167.]]
```

Here we see a coordinates visualization of one set of matching keypoints.

```
Calculating transformation from frame: 10 to frame: 11
[[ 1.00135095 -0.0049859  -2.94561199]
 [ 0.0049859   1.00135095 -2.63641695]]
```

Here we see a visualization of one of the calculated affine transformation estimated, by it's matrix form.

2.3 Cumulative Transformation Calculation (Composite Transformations)

- Input: Set of M amount of 2D affine/rigid transformations.

- Output: Set of M+1 amount 2D composite affine/rigid transformation matrices.

Visualization:

```
Cumulative transformation for i=10 to reference index=0:
[[ 1.00804038e+00  4.32592258e-02  4.55081892e+01]
 [-4.32592258e-02  1.00804038e+00  1.66720178e+01]
 [ 0.00000000e+00  0.00000000e+00  1.00000000e+00]]
```

Here we see a visualization of one of the calculated composite transformation, from frame 10 to frame 0.

2.4 Mosaics Video Creation

- Input: Frames, Transformations, Reference Index, Start Column, End Column, Minimum Threshold for Translation
- Output: Set of panoramas which can be played in order to get a viewpoint/dynamic video.



Example for a left-looking panorama



Example for a right-looking panorama

Implementation Details

1. Implementation

1.1 Keypoint Detection and Feature Matching:

- Keypoints are detected using Harris corners.
- Descriptors are calculated using SIFT.
- Matching is performed using BFMatcher with L2 norm and cross-check to ensure reliability.
- RANSAC (Random Sample Consensus) filters out outliers and computes the best transformation.
- Matches are sorted based on distance.

1.2 Transformation Estimation:

- Use Open-CV function called "estimateAffinePartial2D" to get the transformation matrix, by passing it the matched keypoints we found earlier.

1.3 Cumulative Transformation Calculation (Composite Transformations):

- Regarding some reference index (in our algorithm it is m=0), we calculate the composite transformations as follows (where '$T_{a,b}$' is the transformation from a to b):
  - For i<m we calculate $T_{i,m} = T_{m-1,m} \times \ldots \times T_{i+1,i+2} \times T_{i,i+1}$
  - For i>m we calculate $T_{i,m} = T^{-1}_{m,m+1} \times \ldots \times T^{-1}_{i-2,i-1} \times T^{-1}_{i-1,i}$
  - For i=m we set $T_{i,m}$ = 3x3 I matrix (=np.eye(3))
- Calculations were made using numpy and opencv (np.linalg.inv, np.dot/@ operator, np.vstack if needed to square the matrix and such).

1.4 Mosaics Video Creation:

- Loop over all numbers starting from the Start Column up to the End Column. This loop basically tells us which place, the strip we will take will be from. Just as learnt in class, it can be "left" strips, "middle" strips and so on.
- For each loop, create a blank canvas, by applying the respective transformation on the corners of each corner. We use the minimum and maximum corners of all transformed corners to calculate the canvas size.
- In order to create a viewpoint mosaics video, we crop a vertical strip from each frame by order, when the start index of where we start cropping this strip is the current loop number we're on, meaning 'Start Column' for the first loop. For example, if we loop and want strips starting from column 5, all the strips on the first loop will start on column 5 of each image. We can call these the "left-most" strips for the current viewpoint video we create. The strip widths, are calculated by the X translation of each frame. Here also comes into use the "Minimum Threshold for Translation". If a frame has a translation on the X axis which is lower than this threshold, we skip it and continue. We save computation time and memory by not working on frames that had too low to no translations.
- After cropping each strip, we place it on it's respective place on the blank canvas. At the end of the loop, we have a whole panorama with a viewpoint from the column we started cropping the strips for, in this loop.
- After creating all panoramas in the range of the columns we wanted, i.e. different viewpoints, we combine them all into a video and play them by order. This gives us the wanted effect of a changing viewpoint.

For dynamic videos, the logic is a little different which mainly regards to where we take the strips from.

All in all, in order to implement the different logic required to solve this exercise, I've used function from existing libraries, I've used logic provided by Shmuel in lectures, and logic/implementation I figured out myself such as how to paste onto the canvas and calculate the right position to place the strips we extracted onto the canvas.

Some hyper-parameters/thersholds I've used are for setting what column we start and end taking strips for, offsets for placing strips onto the cavas in the Y/X axis so the panoramas look slightly better, minimum global translation of the frame threshold in order to take it into account in the panorama and not skip it, some more constants for saving the videos such as scaling factor to downsize the frames so the output video weights less and a fps paramater, and such.

The main challenge for me was to figure out what is the logic for taking strips/creating panoramas. There are many ways to do this, such as warping the frames first, taking strips from them and then calculating where to place them on the canvas, or create a canvas and paste strips onto the canvas by backwarping using np.meshgrid and such functions. It took me very long time to solve this by myself and eventually I reached a solution.

Visual Results

First, middle and final frame of the result of the **viewpoint** mosaic:



First, middle and final frame of the result of the **dynamic** mosaic:



First, middle and final frame of the result of the **good self-recorded** mosaic:



First, middle and final frame of the result of the **bad self-recorded** mosaic:



Some flaws we can see in the viewpoint/dynamic mosaics, are the fact that the mosaics are not perfect and lines are not exactly straight and such. This can happen by many reasons, some of them being parallax that happened in the original images, library functions that calculate the keypoints/descriptors/matching/transformations a bit different than expected in this exercise and such.
As for the good video – it is a video I recorded with a horizontal motion of a coat rack, while having little to no rotation and a clean movement on horizon, with also little to no motion on the vertical axis. As we can see, the frames look pretty good, as the video meets the assumptions of the exercise and the algorithm.
As for the bad video – it is a video I recorded of the same coat rack, but with rotations and a lot of scaling. I zoomed in and out the camera while recording the video and moving to the right, and as such, this video does not meet the exercise assumptions. We can see the difference between it and the good video frames pretty easily – we see mirroring, distortions, straight lines became crooked and more bad results, compared to the good video.

Conclusion

In this exercise, the implementation of the stereo mosaicing algorithm demonstrated the ability to create dynamic and viewpoint panoramas by aligning video frames based on rigid transformations and assembling vertical strips onto a unified canvas. The process that I used involved key techniques such as Harris corners, SIFT-based feature matching, RANSAC filtering for transformation estimation, and cumulative transformation calculations for accurate alignment.
Despite challenges such as parallax effects, noise in keypoint matching, and extreme frame translations, the algorithm produced coherent panoramas for videos that adhered to the assumed motion constraints. Videos with smooth and consistent camera motion yielded the best results, while more complex motions led to stitching errors and distortions, highlighting the limitations of the rigid transformation model in handling significant depth variations or rotations.
The results emphasize the importance of preprocessing steps, such as resolution reduction and consistent frame alignment, in enhancing performance and accuracy.

Additionally, well-tuned hyperparameters, such as thresholds for translation and strip widths, played a crucial role in improving robustness.

The creation of both dynamic and viewpoint-changing mosaics demonstrated that this approach is effective in producing visually cohesive panoramas for structured camera movements. Future improvements, such as incorporating optical flow for finer motion adjustments and adaptive strip extraction strategies, could enhance the algorithm's resilience to more complex scenarios. One of the key takeaways from this exercise was the importance of robust feature matching and accurate transformation estimation in achieving high-quality panoramas. During implementation, it became evident that the performance of the mosaicing algorithm heavily depends on the quality of input video frames and the assumptions of the algorithm. For instance, videos with significant parallax or rotational movements introduced errors in the stitching process, such as misaligned strips and distortions. These challenges underscore the limitations of relying solely on rigid transformations, as they fail to account for depth variations or nonlinear camera motions.

Another observation was the impact of preprocessing steps on the algorithm's success. Reducing resolution and frame rate during testing helped speed up computation without compromising the detection of key features. This trade-off was particularly useful when working with high-resolution videos or lengthy clips. Additionally, fine-tuning hyperparameters, such as the minimum translation threshold and strip width, played a crucial role in creating smoother and more visually consistent panoramas.

Testing with both "good" and "bad" videos revealed the algorithm's sensitivity to input quality. In structured scenarios with consistent horizontal or vertical camera motion, the algorithm performed as expected, producing coherent panoramas with minimal distortions. However, in scenarios involving complex motions, such as scaling, rotations, or non-uniform translations, the results showed noticeable artifacts. This highlights an opportunity for future improvement, such as incorporating techniques like optical flow to better handle non-rigid transformations or implementing bundle adjustment to refine alignment.

Beyond the technical implementation, this exercise demonstrated the potential applications of stereo mosaicing in various fields. For example, in virtual reality, such algorithms can be used to stitch environments into panoramic views for immersive experiences. Similarly, in autonomous systems, creating dynamic mosaics could aid in map generation or scene reconstruction. The ability to generate both viewpoint-changing and dynamic mosaics makes this approach versatile and suitable for a wide range of use cases.